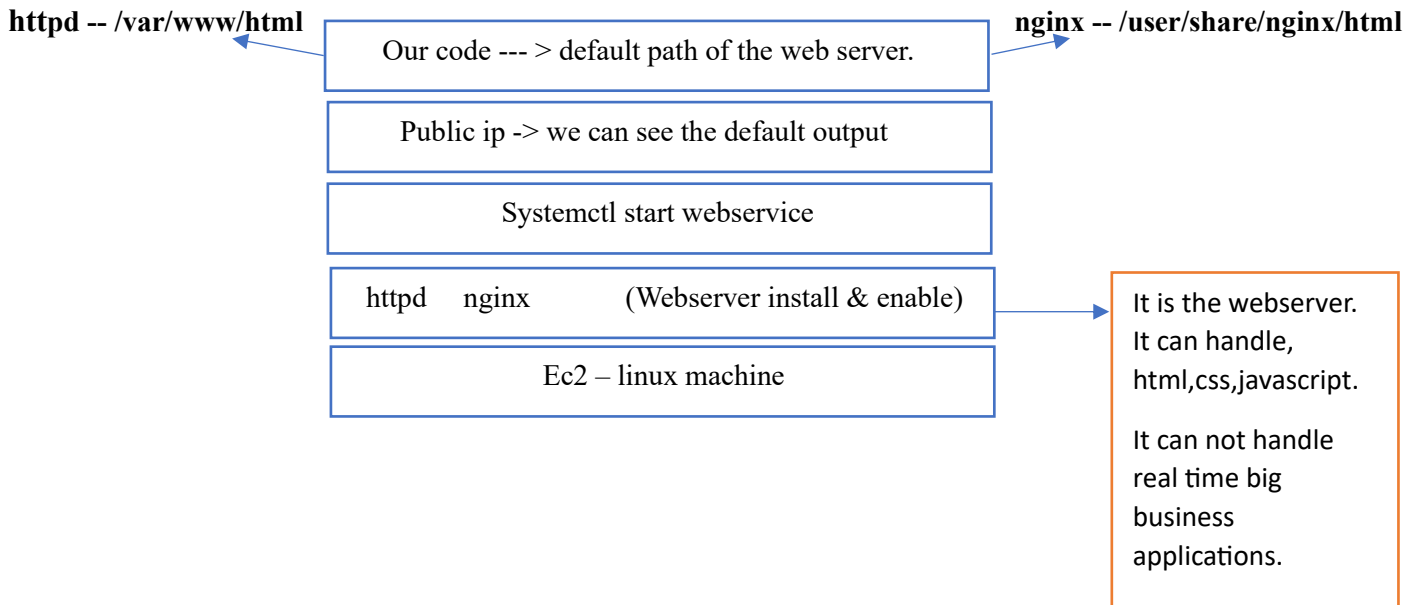


EC2 Web Hosting Our Code using httpd and nginx

Class no 12:

Web hosting:

Bringing our code from our system to the browser to be accessed via internet.



Hands on implementation for httpd Webservice:

Step no:1 Create an AWS Linux Ec2 instance and enable the http / https in network setting.

▼ Network settings [Info](#)

Network | [Info](#)

vpc-08d23a3e5bc324ead

Subnet | [Info](#)

No preference (Default subnet in any availability zone)

Auto-assign public IP | [Info](#)

Enable

Firewall (security groups) | [Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your in

☒ Create security group

☐ Select existing security group

We'll create a new security group called 'launch-wizard-8' with the following rules:

☒ Allow SSH traffic from
Helps you connect to your instance

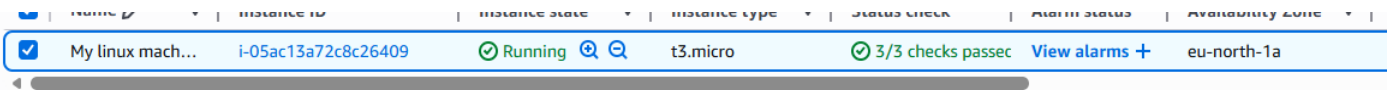
Anywhere
0.0.0.0/0

☒ Allow HTTPS traffic from the internet
To set up an endpoint, for example when creating a web server

☒ Allow HTTP traffic from the internet
To set up an endpoint, for example when creating a web server

⚠ Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security rules with known IP addresses only.

Step no:2 Launch the instance: **My linux machine.**



- (i) Switch user using: **sudo su**
- (ii) Update the machine: **yum update -y**
- (iii) Install the web service: **yum install httpd**

```
/home/ec2-user
[ec2-user@ip-172-31-17-71 ~]$ sudo su
[root@ip-172-31-17-71 ec2-user]# yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-17-71 ec2-user]# yum install httpd
```

Step no:3 once the installation is done.

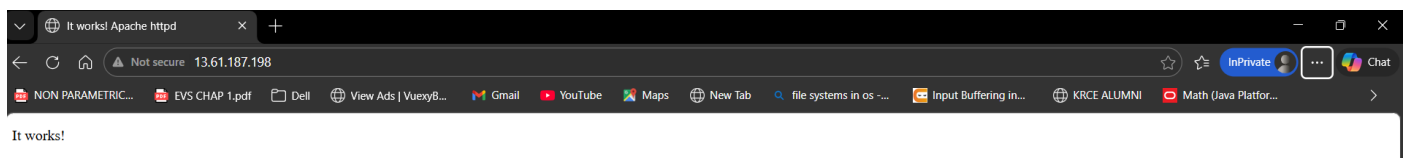
- (i) Check the status of the webservice (httpd): **Systemctl status httpd**
- (ii) Start the webservice (httpd): **Systemctl start httpd**
- (iii) Check status once again to check if it is active.

```
[root@ip-172-31-17-71 ec2-user]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Active: inactive (dead)
     Docs: man:httpd.service(8)
[root@ip-172-31-17-71 ec2-user]# systemctl start httpd
[root@ip-172-31-17-71 ec2-user]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Active: active (running) since Wed 2026-03-25 13:24:19 UTC; 9s ago
     Docs: man:httpd.service(8)
  Main PID: 26211 (httpd)
    Status: "Total requests: 1; Idle/Busy workers 100/0; Requests/sec: 0.111; Bytes served/sec: 70 B/sec"
    Tasks: 177 (limit: 1067)
   Memory: 13.5M
      CPU: 87ms
  CGroup: /system.slice/httpd.service
          └─26211 /usr/sbin/httpd -DFOREGROUND
            └─26236 /usr/sbin/httpd -DFOREGROUND
              └─26237 /usr/sbin/httpd -DFOREGROUND
                └─26246 /usr/sbin/httpd -DFOREGROUND
                  └─26265 /usr/sbin/httpd -DFOREGROUND
```

Step no:4 Copy the public ip address and paste it in the new incognito tab. We can see the default output.

i-05ac13a72c8c26409 (My linux machine)

PublicIPs: **13.61.187.198** PrivateIPs: 172.31.17.71



Step no:5 Deploying our code in the Ec2 instance.

First download the code using the command: **wget -S --content-disposition link_of_the_file**

(https://nithyavel04.github.io/nithyasri-portfolio/).

```
[ec2-user@ip-172-31-17-71 ~]$ wget -S --content-disposition https://nithyavel04.github.io/nithyasri-portfolio/
--2026-03-25 13:55:00-- https://nithyavel04.github.io/nithyasri-portfolio/
Resolving nithyavel04.github.io (nithyavel04.github.io)... 185.199.111.153, 185.199.108.153, 185.199.109.153, ...
Connecting to nithyavel04.github.io (nithyavel04.github.io)|185.199.111.153|:443... connected.
HTTP request sent, awaiting response...
HTTP/1.1 200 OK
Connection: keep-alive
Content-Length: 5096
```

It will be uploaded as a zip file. Check the file by **ls -l** to see it's a zip file or not. If yes, unzip it using: **unzip filename**, if not skip. Here it is not a zip file. It is as a html file.

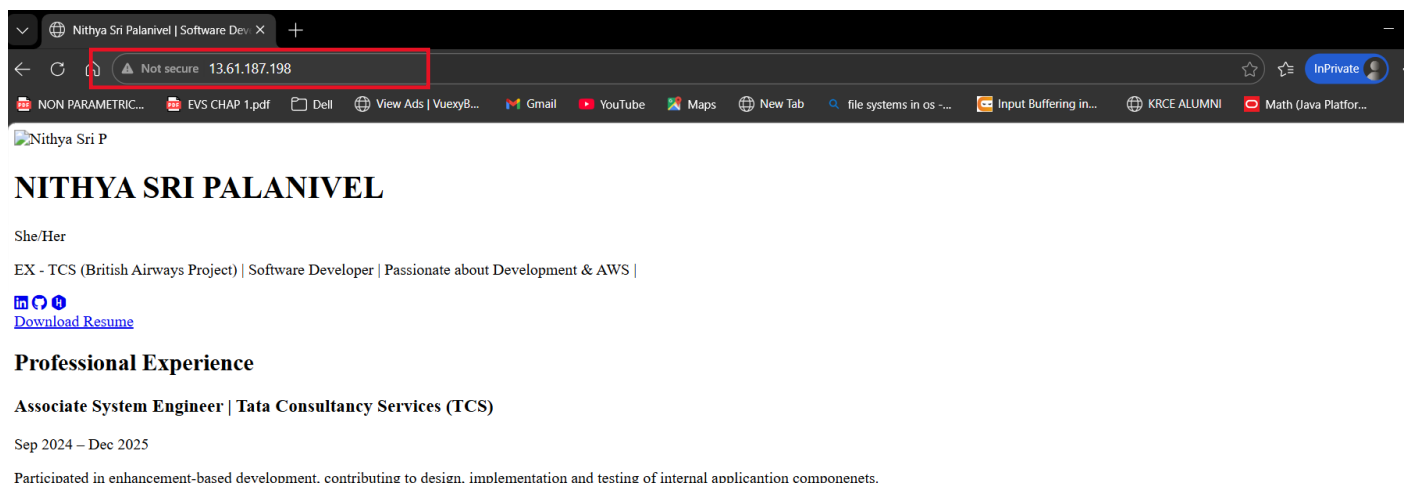
```
Saving to: 'index.html'
index.html 100%[=====]
2026-03-25 14:03:53 (68.7 MB/s) - 'index.html' saved [5096/5096]

[ec2-user@ip-172-31-17-71 ~]$ ls -l
total 8
-rw-r--r--. 1 ec2-user ec2-user 5096 Mar 11 14:02 index.html
[ec2-user@ip-172-31-17-71 ~]$ ls
index.html
```

Move it to the httpd default path using command: **mv * /var/www/html**

```
mv: cannot move 'index.html' to '/var/www/html/index.html': No such file or directory
[ec2-user@ip-172-31-17-71 ~]$ sudo mv * /var/www/html
```

Now **copy the public Ip** and paste in the new incognito browser or refresh the previous incognito browser. We can see our code output there.



Hands on implementation for nginx Webservice:

Set up **Nginx** on your EC2 alongside **httpd**, but we'll run it on a different port so they don't clash.

(Note: We can have two separate instances to run the two web service without changing the port numbers, but it is not advisable).

Step no 1: Ensure we are in the correct home directory using: **pwd**

```
[ec2-user@ip-172-31-17-71 ~]$ pwd
/home/ec2-user
```

Step no 2: Install the Nginx webservice.

- (i) Switch user using: **sudo su**
- (ii) Update the machine: **yum update -y**
- (iii) Install the web service: **yum install nginx**

```
[ec2-user@ip-172-31-17-71 ~]$ sudo su
[root@ip-172-31-17-71 ec2-user]# yum update -y
Last metadata expiration check: 1:02:32 ago on Wed Mar 25 13:17:28 2026.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-17-71 ec2-user]# yum install nginx
Last metadata expiration check: 1:02:41 ago on Wed Mar 25 13:17:28 2026.
Dependencies resolved.
```

Step no:3 once the installation is done.

- (ii) Check the status of the webservice (httpd): **Systemctl status nginx**
- (iv) Start the webservice (httpd): **Systemctl nginx**

Check status once again to check if it is active.

```
[root@ip-172-31-17-71 ec2-user]# systemctl status nginx
○ nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)
   Active: inactive (dead)
[root@ip-172-31-17-71 ec2-user]# systemctl start nginx
Job for nginx.service failed because the control process exited with error code.
See "systemctl status nginx.service" and "journalctl -xeu nginx.service" for details.
[root@ip-172-31-17-71 ec2-user]# systemctl status nginx
* nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)
   Active: failed (Result: exit-code) since Wed 2026-03-25 14:23:28 UTC; 8s ago
     Process: 30830 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 30831 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 30832 ExecStart=/usr/sbin/nginx (code=exited, status=1/FAILURE)
```

The failure you're seeing (Job for nginx.service failed) usually means **Nginx is trying to bind to port 80**, but **httpd** is already running there. Since you want both Apache and Nginx on the same EC2, you need to change Nginx's port.

Step no:4 How to fix it?

Open the main config file: **sudo nano /etc/nginx/nginx.conf**

```

GNU nano 8.3
# For more information on configuration, see:
#   * Official English Documentation: http://nginx.org/en/
#   * Official Russian Documentation: http://nginx.org/ru/

user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /run/nginx.pid;

# Load dynamic modules. See /usr/share/doc/nginx/README.dynamic
include /usr/share/nginx/modules/*.conf;

events {
    worker_connections 1024;
}

http {
    log_format main '$remote_addr - $remote_user [$time_local] $status $body_bytes_sent "$http_referer" "$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;

    sendfile on;
    tcp_nopush on;
    keepalive_timeout 65;
    types_hash_max_size 4096;

    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # Load modular configuration files from the /etc/nginx/conf.d directory. See http://nginx.org/en/docs/nginx_core_module.html#conf
    # See http://nginx.org/en/docs/nginx_core_module.html#conf
    # For more information

```

In Nginx, you often see two listen lines:

- listen 80; (for IPv4)
- listen [::]:80; (for IPv6)

Change both as :

- listen **8080**;
- listen [::]:**8080**;

Save: **cntrl + o** -> enter and exit: **cntrl + x**

```

include /etc/nginx/conf.d/*.conf;

server {
    listen      8080;
    listen     [::]:8080;
    server_name _;
    root       /usr/share/nginx/html;

```

Step no:5 After saving, don't forget to run: **sudo nginx -t** . Ensure it is successful.

```
[root@ip-172-31-17-71 ec2-user]# sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Step no:6 Restart the nginx server: **sudo systemctl restart nginx**. Also check the status is active.

```
[root@ip-172-31-17-71 ec2-user]# sudo systemctl restart nginx
[root@ip-172-31-17-71 ec2-user]# systemctl status nginx
• nginx.service - The nginx HTTP and reverse proxy server
  Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)
  Active: active (running) since Wed 2026-03-25 14:37:27 UTC; 22s ago
```

Step no:7 Since you configured Nginx to listen on **port 8080**, you need to explicitly allow that port in the inbound rules.

Otherwise, even though Nginx is running, traffic from outside won't reach it. To fix it,

Go to **Security groups** - In the **Edit inbound rules** page:

- Click **Add rule**.
- Choose **Custom TCP**.
- Set **Port range** to 8080.
- Set **Source** to 0.0.0.0/0 (or restrict to your IP for better security).
- Save rule.

EC2 > Security Groups > sg-0941fcd3b4939d3174 - launch-wizard-8 > Edit inbound rules

Edit inbound rules Info

Inbound rules control the incoming traffic that's allowed to reach the instance.

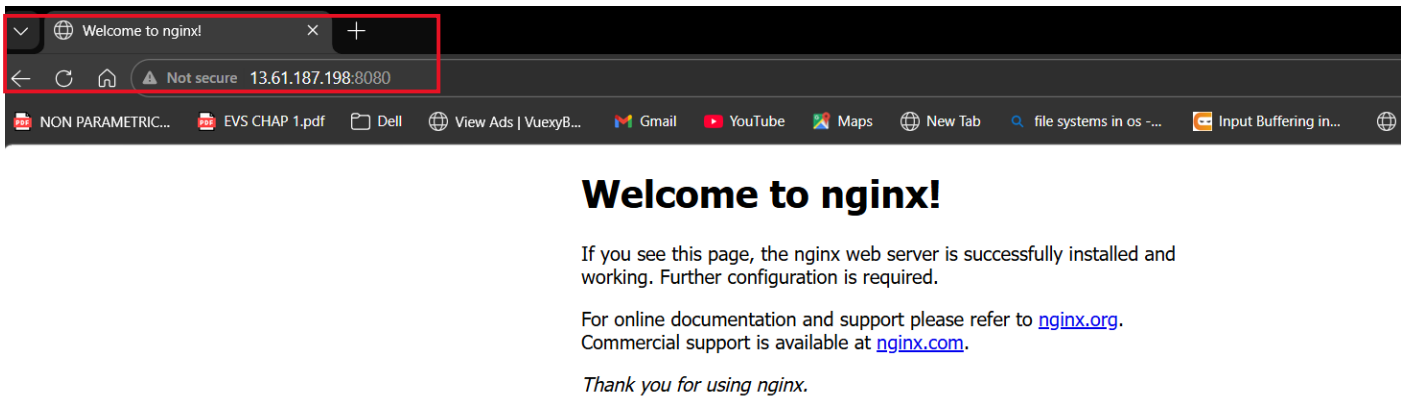
Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
sgr-0d786e4594f8770fd	HTTP	TCP	80	Custom	0.0.0.0/0	Delete
sgr-00362d6e93286ea57	SSH	TCP	22	Custom	0.0.0.0/0	Delete
sgr-04570c196e59dc2e1	HTTPS	TCP	443	Custom	0.0.0.0/0	Delete
-	Custom TCP	TCP	8080	Anyw...	0.0.0.0/0	Delete

[Add rule](#)

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

[Cancel](#) [Preview changes](#) [Save rules](#)

Step no:8 Copy the public Ip address and paste it as **public ip:8080** in new private tab, we can see the default output of nginx.



At the same time if we paste the **public Ip** alone we can see the httpd code output.



Step no:9 If we want to add our code on the nginx webservice, download the code using:

wget -S --content-disposition link_of_the_file

```
[root@ip-172-31-17-71 ec2-user]# wget -S --content-disposition https://templatemo.com/tm-614-quantix-saas
--2026-03-25 14:51:36-- https://templatemo.com/tm-614-quantix-saas
Resolving templatemo.com (templatemo.com)... 172.67.139.186, 104.21.94.200, 2606:4700:3031::ac43:8bba, ...
Connecting to templatemo.com (templatemo.com)|172.67.139.186|:443... connected.
HTTP request sent, awaiting response...
HTTP/1.1 200 OK
```

Step no:10 It will be uploaded as a zip file. Check the file by **ls -l** to see it's a zip file or not. If yes, unzip it using: **unzip filename**, if not skip. Here it is not a zip file. It is as a **html file** named as **tm-614-quantix-saas**.

```
[root@ip-172-31-17-71 ec2-user]# ls
tm-614-quantix-saas
[root@ip-172-31-17-71 ec2-user]# file tm-614-quantix-saas
tm-614-quantix-saas: HTML document, UTF-8 Unicode text, with very long lines, with CRLF, LF line terminators
```

Move it to Nginx's root directory: **sudo mv tm-614-quantix-saas /usr/share/nginx/html/index.html**

Paste **the public ip:8080** in a new private tab, we can see our code output.

